

Assignments:

1. Install and configure Tomcat version 9 on your Linux machine with the Manager and Host Manager sections enabled.

Ans:

→ sudo yum install java-21-openjdk-devel -y

→ java -version

Output: openjdk version "21.0.10"

→ sudo useradd -m -U -d /opt/tomcat -s /bin/false tomcat

→ sudo yum install wget -y

→ cd /tmp

→ wget <https://dlcdn.apache.org/tomcat/tomcat-11/v11.0.18/bin/apache-tomcat-11.0.18.tar.gz>

→ sudo tar xf /tmp/apache-tomcat-11.0.18.tar.gz -C /opt/tomcat --strip-components=1

→ rm apache-tomcat-11.0.18.tar.gz

→ sudo chown -R tomcat: /opt/tomcat

→ sudo chmod -R 755 /opt/tomcat

→ sudo vi /etc/systemd/system/tomcat.service

[Unit]

Description=Apache Tomcat Web Application Container

After=network.target

[Service]

Type=forking

User=tomcat

Group=tomcat

Environment="JAVA_HOME=/usr/lib/jvm/jre"

Environment="CATALINA_PID=/opt/tomcat/temp/tomcat.pid"

Environment="CATALINA_HOME=/opt/tomcat"

Environment="CATALINA_BASE=/opt/tomcat"

Environment="CATALINA_OPTS=-Xms512M -Xmx1024M -server
-XX:+UseParallelGC"

Environment="JAVA_OPTS=-Djava.awt.headless=true

-Djava.security.egd=file:/dev/./urandom"

ExecStart=/opt/tomcat/bin/startup.sh

ExecStop=/opt/tomcat/bin/shutdown.sh

[Install]

WantedBy=multi-user.target

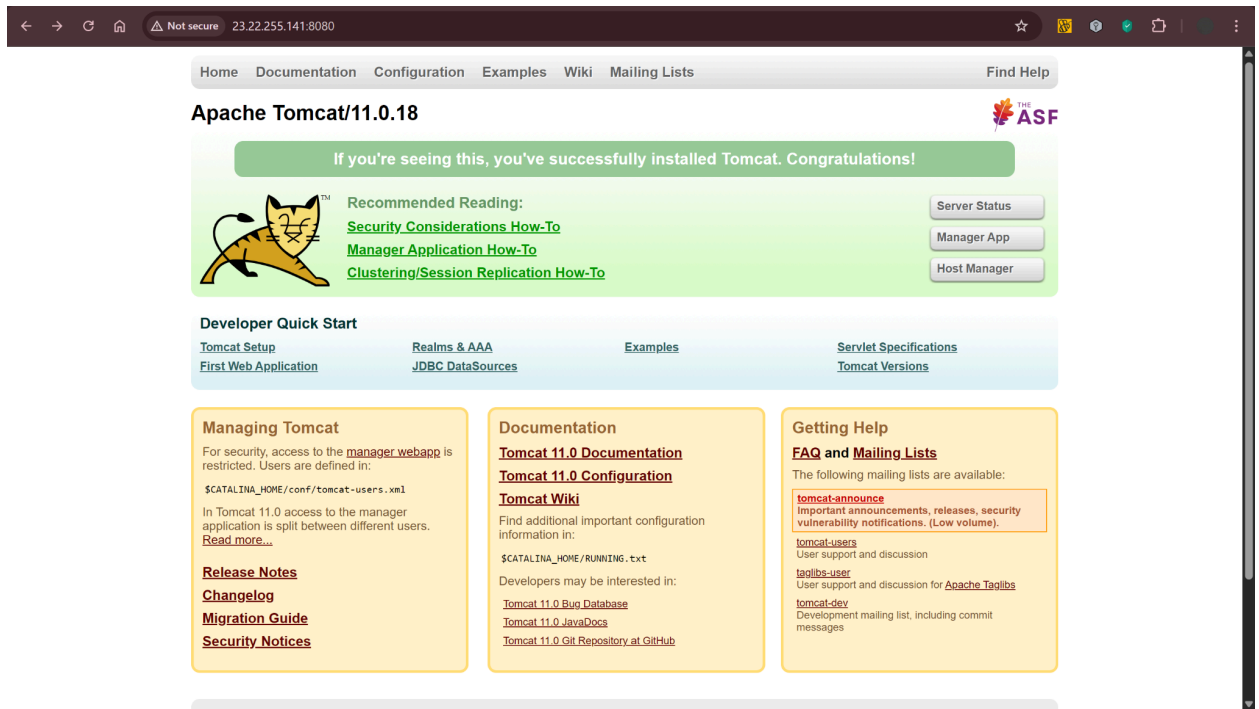
→ :wq

→ sudo systemctl daemon-reload

→ sudo systemctl start tomcat

→ sudo systemctl enable tomcat

Output: Access the Tomcat dashboard using public IP with port 8080:
http://23.22.255.141:8080/ -> (do not use https, it won't work. Use http)



Enable the manager and host manager:

→ sudo vi /opt/tomcat/conf/tomcat-users.xml

```
<role rolename="manager-gui"/>
<role rolename="manager-script"/>
<role rolename="manager-jmx"/>
<role rolename="manager-status"/>
<role rolename="admin-gui"/>
<role rolename="admin-script"/>
<user username="admin" password="admin"
roles="manager-gui,admin-gui"/>
<!-- manager-gui,admin-gui -Roles assigned to the user, allowing access to both
Manager and Host Manager interfaces -->
```

→ :wq

→ sudo vi /opt/tomcat/webapps/manager/META-INF/context.xml

→ comment on the Valve className line.

i.e., Before:

```
<Context antiResourceLocking="false" privileged="true" >
<CookieProcessor
className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
sameSiteCookies="strict" />
```

```

<Valve className="org.apache.catalina.valves.RemoteAddrValve"
allow="\127\.d+\.d+\.d+|::1|0:0:0:0:0:0:1" />
<Manager
sessionAttributeValueClassNameFilter="java\.lang\.(?:Boolean|Integer|Long|Num
ber|String)|org\.apache\.catalina\.filters\.CsrfPreventionFilter\$LruCache(?:\$1)?|j
ava\.util\.(?:Linked)?HashMap"/>
</Context>

```

After:

```

<Context antiResourceLocking="false" privileged="true" >
<CookieProcessor
className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
sameSiteCookies="strict" />
<!-- <Valve className="org.apache.catalina.valves.RemoteAddrValve"
allow="\127\.d+\.d+\.d+|::1|0:0:0:0:0:0:1" />
-->
<Manager
sessionAttributeValueClassNameFilter="java\.lang\.(?:Boolean|Integer|Long|Num
ber|String)|org\.apache\.catalina\.filters\.CsrfPreventionFilter\$LruCache(?:\$1)?|j
ava\.util\.(?:Linked)?HashMap"/>
</Context>

```

→ sudo vi /opt/tomcat/webapps/host-manager/META-INF/context.xml
Repeat the above step:

After:

```

<Context antiResourceLocking="false" privileged="true" >
<CookieProcessor
className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
sameSiteCookies="strict" />
<!-- <Valve className="org.apache.catalina.valves.RemoteAddrValve"
allow="\127\.d+\.d+\.d+|::1|0:0:0:0:0:0:1" />
-->
<Manager
sessionAttributeValueClassNameFilter="java\.lang\.(?:Boolean|Integer|Long|Num
ber|String)|org\.apache\.catalina\.filters\.CsrfPreventionFilter\$LruCache(?:\$1)?|j
ava\.util\.(?:Linked)?HashMap"/>
</Context>

```

→ sudo systemctl restart tomcat

Output: Now you can access <http://23.22.255.141:8080/> and click on manager app and host-manager app button or else use: <http://23.22.255.141:8080/manager/html> and <http://23.22.255.141:8080/host-manager/html>

Enter user: **admin**

Password: **admin**

Screenshot of manager:

Tomcat Web Application Manager

Message: OK

Manager

[List Applications](#) [HTML Manager Help](#) [Manager Help](#) [Server Status](#)

Applications

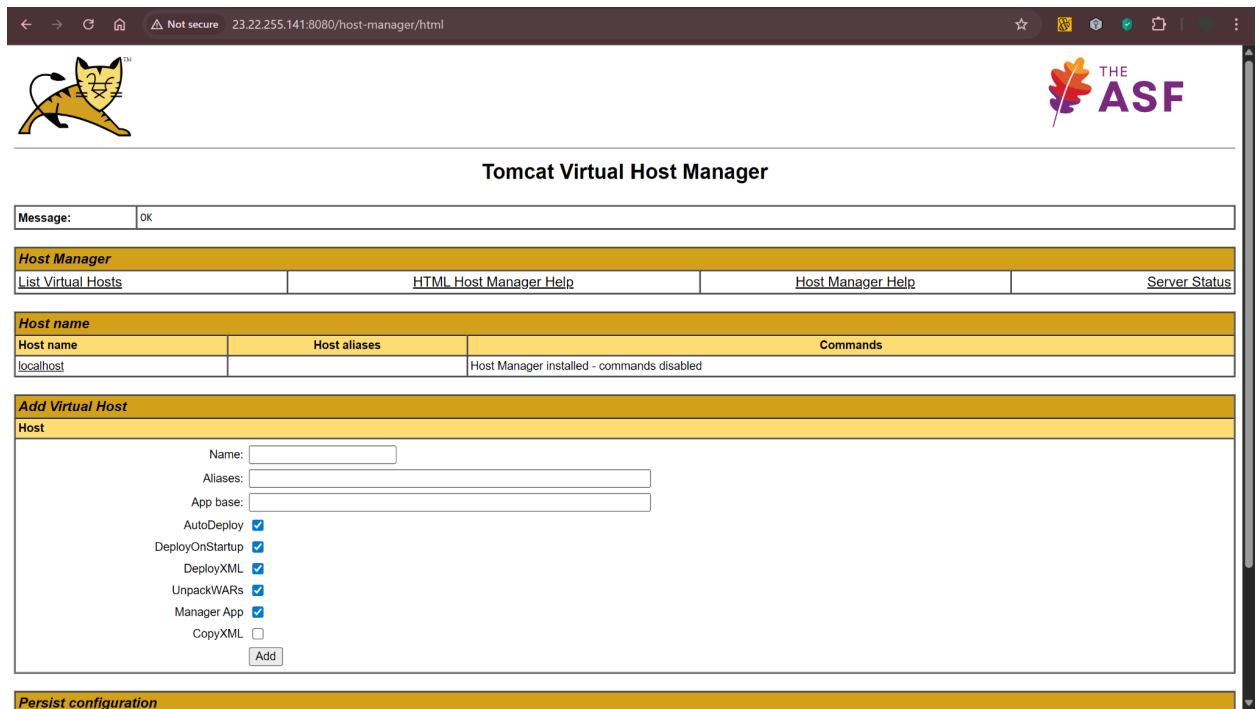
Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy

Deploy directory or WAR file located on server

Context Path:

Screenshot of Host manager:



2. Change the default Tomcat port to 9050.

Ans:

→ `sudo vi /opt/tomcat/conf/server.xml`

Before:

```
<Connector port="8080" protocol="HTTP/1.1"
connectionTimeout="20000"
redirectPort="8443" />
```

After:

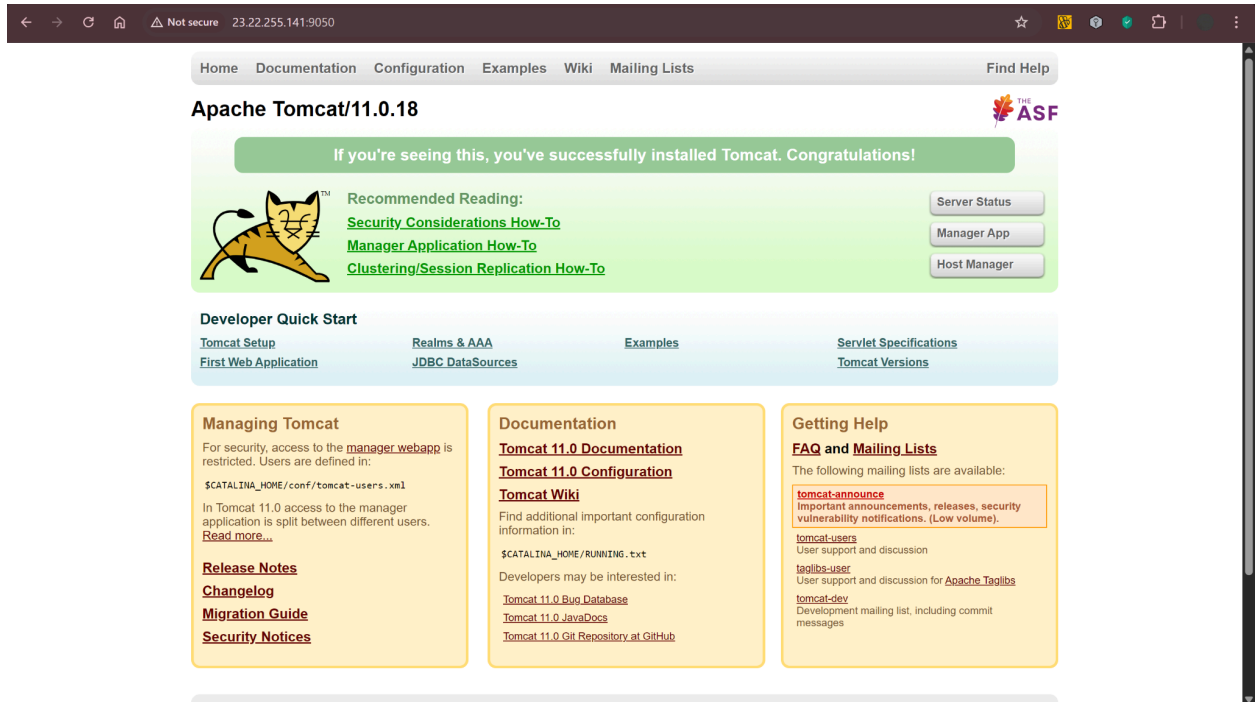
```
<Connector port="9050" protocol="HTTP/1.1"
connectionTimeout="20000"
redirectPort="8443" />
```

→ `:wq`

→ `sudo systemctl restart tomcat`

Output:

`http://23.22.255.141:9050/`



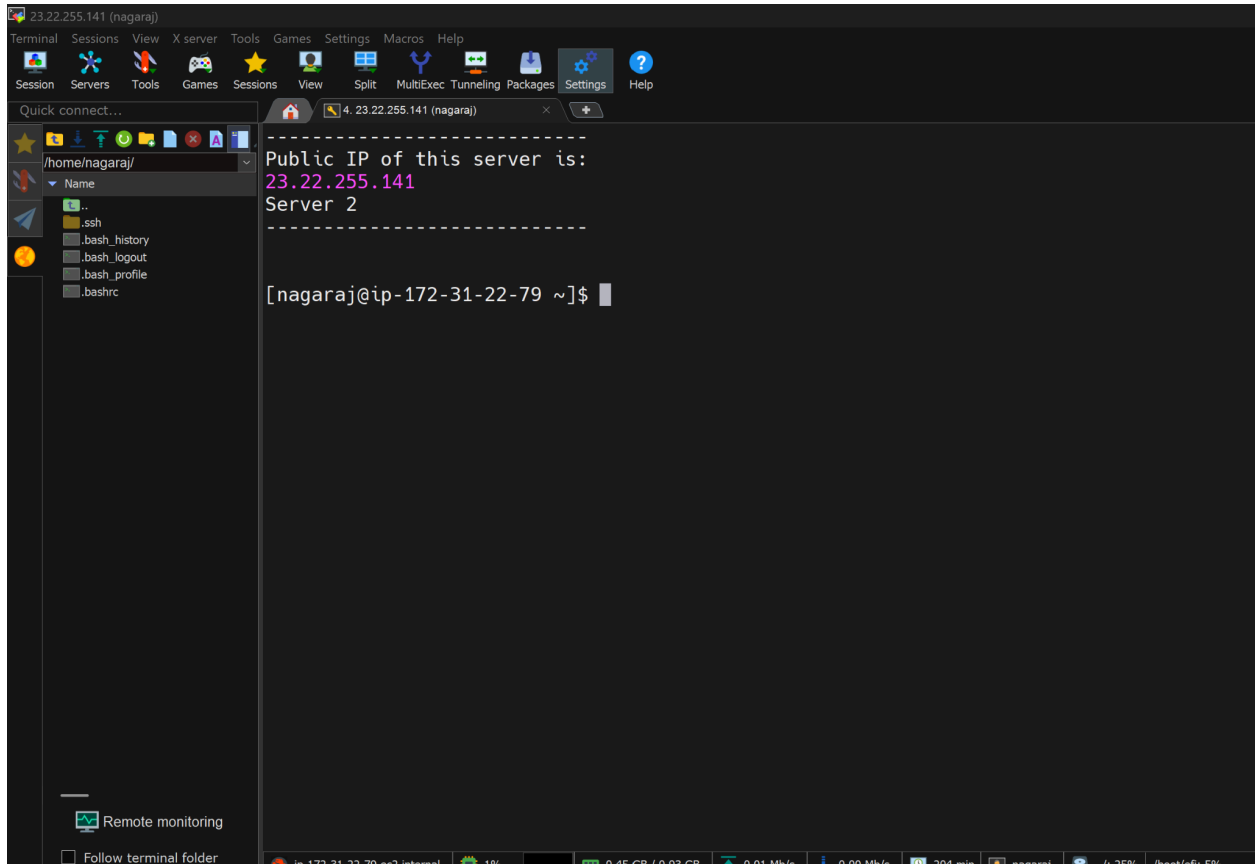
3. Create a new user and configure the necessary settings for secure login via Putty or other remote access tools.

Ans:

- Created a new server and logged in via ssh client and copied the public key from authorized_keys file.
- Created new user using this command:
 - `sudo useradd nagaraj`
 - `sudo su - nagaraj`
 - `mkdir .ssh`
 - `touch .ssh/authorized_keys`
 - paste the public key to authorized_keys file.
 - Open ssh client and enter server IP, username and private key.

Username as: nagaraj

→ Able to login to ssh using the new user.



4. Establish passwordless connection between two linux servers

Ans:

→ Create 2 servers

→ login to 1st server and generate private key and public key using ssh-keygen command and copy the public key to 2nd server and now will be able to access 2nd server via first server.

→ cd .ssh

→ ssh-keygen -t rsa

→ ls

authorized_keys id_rsa id_rsa.pub

→ cat id_rsa.pub

Copy the content of id_rsa.pub.

→ Login to the 2nd server via ssh.

→ echo "copy of public key from 2nd server" >> authorized_keys

ex:

echo "ssh-rsa

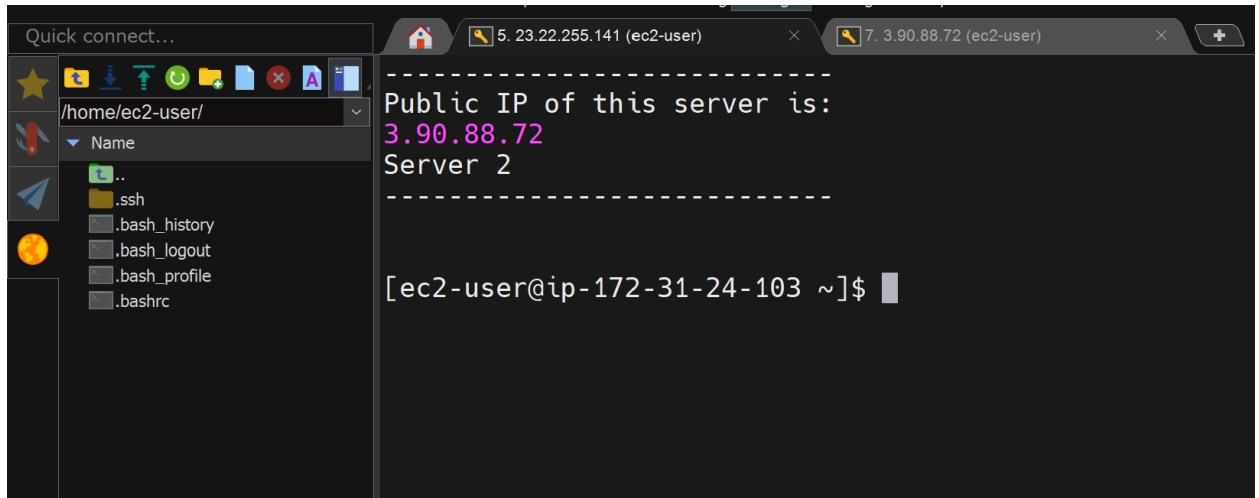
AAAAB3NzaC1yc2EAAAADAQABAAQGCvoXVGUdkPyZBbvtFAVYk6ycasNDV5OLOOdjlCnHKOrjSqM
xEZ77Uvb4ApP+0uWKiqAeH134u3Gi2dcg/nq/tglviiiNQpSJLKyewkBS7KF0tNnL68p9WnKwlp2opcGsvmO0
YYce2Bd6iXyEvO7BsSkOsWO6C82H9IHrD+REtY5fWAXQeWo9ahp9FqwP+oVl68oAyScfaufros8MtdzotNo

wRDAzx1fSjuEYum5z1wPa80gb3Kv1svO/aDubaXp0QNI3ndIRD8VfjrkeRocNVRpeFQ/qcTmGmYXlwkOUsh
oh5gJeEEYzd93Vgr0u5X6yYRwOeaZF7yxtJp7Otup1QGolp7UkPZH/WxQ2vjE1W7Lh2cezeJPDMw33cRi1
kHKNoGZsEssLuckQdDpxq5FMJ47Az7NFNQnp+UqClw+S5Y71flqktjyMYWiERJo58azSi96ZXIKJPh8IBzAl
w9RCWiJ28HzTzo5RACQZkMCIB+5KiA79ey3Vsi5ZSPY4c= ec2-user@ip-172-31-22-79.ec2.internal" >>

authorized_keys

→ From the first server access the 2nd server

-> ssh ec2-user@3.90.88.72



5. Create a new user account on your Linux machine and grant them sudo privileges.

Ans:

→ Created new user using this command:

sudo useradd nagaraj

→ sudo visudo /etc/sudoers

→ nagaraj ALL=(ALL) NOPASSWD: ALL

→ sudo su - nagaraj

→ sudo yum install git -y

```
[root@ip-172-31-22-79 ec2-user]# sudo su nagaraj
[nagaraj@ip-172-31-22-79 ec2-user]$ cd
[nagaraj@ip-172-31-22-79 ~]$
[nagaraj@ip-172-31-22-79 ~]$
[nagaraj@ip-172-31-22-79 ~]$ sudo yum install git
Updating Subscription Management repositories.
Unable to read consumer identity

This system is not registered with an entitlement server. You can use "rhc" or "subscription-manage

Last metadata expiration check: 2:41:08 ago on Sat Feb 7 12:58:53 2026.
Dependencies resolved.
=====
Package                                Architecture           Version                Repo
=====
Installing:
git                                    x86_64                 2.47.3-1.el10_0        rhe
Installing dependencies:
git-core                             x86_64                 2.47.3-1.el10_0        rhe
git-core-doc                         noarch                 2.47.3-1.el10_0        rhe
perl-Error                           noarch                 1:0.17029-18.el10      rhe
perl-File-Find                       noarch                 1.44-512.2.el10_0      rhe
perl-Git                             noarch                 2.47.3-1.el10_0        rhe
perl-TermReadKey                     x86_64                 2.38-24.el10           rhe
perl-lib                             x86_64                 0.65-512.2.el10_0      rhe
=====
Transaction Summary
=====
Install 8 Packages

Total download size: 8.1 M
Installed size: 37 M
Is this ok [y/N]: y
Downloading Packages:
```

6. Use the 'scp' command to securely copy files and directories between Linux machines without password

Ans:

→ Created 2 servers and created public and private key using ssh-keygen
→ Login to 1st server via ssh
→ cd .ssh
→ ssh-keygen -t rsa
→ cat id_rsa.pub

Copy the content shown in public key file

→ login to 2nd server via ssh
→ paste the public key content to authorized_key file in .ssh folder
→ Copy the file to 2nd server from first server using scp command:
scp <filename> ec2-user@<2nd-server-IP>:<destination_path>
scp f1.txt ec2-user@3.90.88.72:/home/ec2-user/

7. How can you list the contents of a tar.gz file and extract a particular file

Ans:

→ touch f1 f2 f3 (created 3 files)
→ tar -cvzf backup.tar.gz f1 f2 f3 (compress file and folders to [.tar.gz](#))
→ Now I will delete f2 from the server and will restore it from the archive.

→ rm f2

→ ls

Output: backup.tar.gz f1 f3

(f2 file is missing)

→ tar -tzf backup.tar.gz

(lists all the files in the archive)

Output:

f1

f2

f3

→ tar -xzf backup.tar.gz f2

(extract file f2 from the archive)

→ ls

Output: backup.tar.gz f1 f2 f3

8. How can you compress a directory or file into a tar archive on Linux

Ans:

→ tar -cvf backup.tar ./

9. Write a command to synchronise files and directories between 2 local folders

Ans:

rsync -a /source/path/ /destination/path/ - -> (-a, --archive: Archive mode; preserves symbolic links, permissions, timestamps, and recursively copies Directories.)

→ mkdir -p d1/f1 d2

→ ls -R

Output:

..:

d1 d2

./d1:

f1

./d1/f1:

./d2:

→ rsync -a d1/f1 d2

→ ls -R

Output:

..:

d1 d2

./d1:

f1

./d1/f1:

./d2:

f1

./d2/f1:

10. What is load average and swap memory in Linux

Ans:

Load average: The load average is a set of three numbers that represent the average number of processes waiting to run over different periods of time (typically over the last 1, 5, and 15 minutes). It's a measure of system load or workload.

ex:

load average: 0.12, 0.23, 0.45

Swap memory (or swap space) is a designated area on disk that the operating system used as virtual memory when it runs out of physical RAM (Random Access Memory). Swap memory allows the system to move less frequently accessed memory pages out of physical RAM and onto disk, freeing up RAM for more active processes.

11. What are the important system directories in linux and what are they generally used for [Ex: var, home etc]

Ans:

/root:

- Home directory for the root user (superuser/administrator). It contains configuration files specific to the root user.

/home:

- User home directories are usually located here. Each user has their own subdirectory within /home where they can store personal files and configuration settings.

/opt (Optional):

- Typically used for installing optional software packages that are not part of the core operating system distribution.

/tmp (Temporary):

- A directory for temporary files created by various programs. Files in /tmp are usually deleted automatically upon reboot.

/bin (Binary Binaries):

- Contains essential executable binaries (programs) that are required for system operation and for booting into single-user mode.

/boot:

- Contains the kernel and files needed for booting the operating system, including bootloader configuration files (e.g., GRUB configuration).

/dev (Devices):

- Contains device files representing physical and virtual devices attached to the system, such as hard drives, USB devices, serial ports, etc.

/etc (Editable Text Configuration):

- Contains system-wide configuration files and scripts that are used by various programs. Configuration files for the system and installed applications are typically stored here.

/lib (Libraries):

- Contains shared libraries required by the essential binaries in /bin and /sbin. These libraries are crucial for the functioning of core system programs.

/mnt (Mount):

- A standard location where temporary filesystems (such as external drives, network shares) are mounted manually.

/proc (Process Information):

- A virtual filesystem that provides detailed information about system processes and kernel parameters. It's used by many system utilities to obtain runtime system information.

/run:

- A temporary filesystem (tmpfs) that stores runtime information about the system since the last boot. It often contains PID files and sockets.

/sbin (System Binaries):

- Contains essential system administration binaries that are crucial for system maintenance and management. These binaries are typically used by the root user.

/srv (Service):

- Used to store data for services provided by the system. It often contains data that is served by the system, such as websites, FTP data, etc.

/sys (System):

- A virtual filesystem that exposes kernel-related information and configuration options. It's used for interacting with the kernel and adjusting kernel parameters at runtime.

/usr (Unix System Resources):

- Contains the majority of user utilities and applications. It's typically read-only after the system is installed, with the exception of /usr/local which is used for locally installed software.

/var (Variable):

- Contains variable data files—files that are expected to grow in size during normal operation of the system. This includes log files, spool files (e.g., mail), and temporary files from user sessions.

12. What are the package installers for Redhat, Ubuntu, Debian, CentOS, Alpine

Ans:

1. Red Hat, CentOS: yum (older versions), dnf (newer versions)
2. Debian, Ubuntu: apt
3. Alpine Linux: apk

13. Configure Gmail on your Linux machine using Postfix/Sendmail to enable sending emails from your Gmail Account through the command line

Ans:

-> sudo yum install postfix s-nail cyrus-sasl-plain -y

Note: First you need to get the app password from gmail for smtp settings. Go to manage your google account >> search for "App password" from the search bar as it won't be visible in the dashboard. Then you can set the app password and use it.

→ vi ~/.mailrc

Set MTA to use Gmail's SMTP server with STARTTLS

set mta=smtp://smtp.gmail.com:587

Authentication settings

set smtp-auth=login

set smtp-auth-user=nagarajkamath602@gmail.com

set smtp-auth-password=gwdyxydvtdnmlhce

Email settings

set from="nagarajkamath602@gmail.com"

Enable TLS

set ssl-verify=ignore

set smtp-use-starttls

→ :wq

→ echo "This mail from my server" | s-nail -s "Server email" nagarajkamath602@gmail.com

 Compose

 **Inbox** 4

-  Starred
-  Snoozed
-  Sent
-  Drafts
-  Purchases
-  More

Labels +



Server email Inbox x



nagarajkamath602@gmail.com

to me ▾

This mail from my server

 Reply

 Forward

